Correttezza e analisi delle performance

Materia	Algoritmi
Anno	Secondo Anno
Data	@October 3, 2024
Studiato	Done

Dato un problema, è possibile che vi siano più algoritmi che lo risolvano correttamente. Tuttavia, è preferibile scegliere quello con performance migliori.
Misurare le performance di un algoritmo significa misurare le risorse che utilizza, quali:

- **memoria**;
- **tempo computazionale** (o **runtime**);
- **larghezza di banda di comunicazione**;
- **energia consumata**.

Sicuramente il **runtime** è tra le più importanti. Un algoritmo, inoltre, potrebbe essere non eseguito su un computer, quindi utilizzare una diversa tecnologia.
Quando descriviamo un algoritmo e lo analizziamo, è importante **stabilire il modello tecnologico considerato**. Utilizzeremo il modello **Random Access Machine (RAM)**. Assumeremo inoltre che gli algoritmi sono formalizzati come programmi informatici scritti in pseudo-codice. Lo pseudo-codice è comodo perché possiamo tradurlo poi facilmente in qualsiasi linguaggio di programmazione, e quindi permette **generalizzazione dell'analisi**.

Il modello RAM ha elementi in comune con tutti i computer che utilizziamo. Le sue caratteristiche, infatti, sono le seguenti:

- le istruzioni di un algoritmo sono eseguite in maniera **sequenziale**, ossia non avremo istruzioni eseguite in parallelo;
- **istruzioni elementari e accesso ai dati impiegano tempo costante**. Le istruzioni elementari sono le seguenti:
  - **operazioni aritmetiche**, quali somma, sottrazione, moltiplicazione, divisione, modulo, arrotondamento per difetto ed eccesso;
  - **spostamento dei dati**, quindi upload, save e copy;
  - **controllo**, quindi branching condizionale e non condizionale, chiamata di subroutine e return.
- diversi formati di dati:

    - **interi;**
    - **floating point;**
    - **caratteri**.

    Il tipo booleano viene incluso negli interi, dove falso corrisponde a 0, e vero a diverso da 0. Inoltre, ogni stringa di dati può essere codificata da un numero limitato di bit.

- **Non esistono gerarchie di memoria** nel modello RAM che consideriamo.

La nozione di **misura dell'input** dipende dal problema (task) considerato, e dalle strutture dati che noi impieghiamo. Ad esempio, se abbiamo un array, misureremo il numero dei suoi elementi, se abbiamo un albero conteremo i suoi nodi, e per un grafo considereremo sia il numero di nodi sia di archi.

Il **tempo computazionale di un algoritmo** è il **numero di istruzioni elementari e accessi ai dati eseguiti in funzione della misura dell'input**.

Ad esempio, consideriamo l'insertion sort:

```
for i=2 to n:
   key = A[i]
   j = i - 1
   while j > 0 and A[j] > key
      A[j+1] = A[j]
      j = j - 1
   A[j+1] = key
```

La prima istruzione ha costo c_1 e si ripete n volte ($n-1$ volte per il ciclo, 1 volta per vedere se la variabile è superiore ad n). La seconda, terza e ultima istruzione si ripetono $n-1$ volte e costo c_2, c_3, c_7. Il ciclo while viene ripetuto $\sum_{i=2}^{n} t_i$, perché non è un tempo costante, mentre le istruzioni del while vengono ripetute $t_i - 1$ volte. Quindi diremo che:

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + c_4 \sum_{i=2}^{n} t_i + c_5 \sum_{i=2}^{n} (t_i - 1) + c_6 \sum_{i=2}^{n} (t_i - 1) + c_7(n-1)$$

Analizziamo il **best case**, ossia il caso migliore. Il caso migliore è ovviamente il caso in cui l'array è già ordinato, quindi $t_i = 1 \ \ \forall i \in \{ 2, ..., n \}$. Riscrivendo in funzione di ciò $T(n)$, avremo:

$$T(n) = c_1 n + c_2(n-1) + c_3(n-1) + c_4(n-1) + c_5 \cdot 0 + c_6 \cdot 0 + c_7(n-1)$$

La funzione quindi è **lineare**, perché possiamo raccogliere n è ottenere una forma del tipo $an + b$.

Nel **worst case**, ossia il caso peggiore, avremo che l'array è ordinato in maniera inversa, quindi $t_i = i \ \ \forall i \in \{ 2, .., n \}$. Ricordando che $\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$, possiamo dire che $\sum_{i=2}^{n} i = \frac{n(n+1)}{2} - 1$. Mentre $\sum_{i=2}^{n} i - 1$ si può scrivere come la somma da i a $n-1$. Quindi il risultato di $T(n)$ sarà:

$$T(n) = (\frac{c_4}{2} + \frac{c_5}{2} + \frac{c_6}{2})n^2 + (c_1 + c_2 + c_3 + \frac{c_4}{2} - \frac{c_5}{2} - \frac{c_6}{2} + c_7)n - c_2 - c_3 - c_4 - c_7$$

Quindi un tempo **quadratico**.

Analizzando gli algoritmi, ci interessa sapere **l'ordine di grandezza** del tempo computazionale. Diremo che
un algoritmo è più efficiente di un altro se il runtime nel caso peggiore cresce più lentamente al crescere della lunghezza dell'input.

Notazione asintotica

Notazione Big O

La notazione O (**Big O**) denota un **limite superiore** (**upper bound**) al comportamento asintotico di una funzione. Ad esempio, sia $f(n) = 7n^3 + 2n - 2n + 6$. Questa funzione **non cresce più rapidamente di** $8n^3$. Quindi possiamo dire che $f(n) = O(n^3)$. Ma possiamo anche dire che $f(n) = O(n^5)$, $f(n) = O(n^8)$ e così via.

Definiamo quindi $O(g(n))$ nel seguente modo:

$$O(g(n)) = \{\, f(n) \mid \exists c \in \mathbb{Q} > 0, \exists n_0 \in \mathbb{N} : \forall n \geq n_0, 0 \leq f(n) \leq c \cdot g(n) \,\}$$

Notazione Big Omega

La notazione Ω (**Big Omega**) denota un **limite inferiore** (**lower bound**) al comportamento asintotico di una funzione. Riprendendo l'esempio precedente, possiamo dire che $f(n) = \Omega(7n^3)$ e anche $f(n) = \Omega(n)$ e così via. Definiamo $\Omega(g(n))$ nel seguente modo:

$$\Omega(g(n)) = \{\, f(n) \mid \exists c \in \mathbb{Q} > 0, \exists n_0 \in \mathbb{N} : \forall n \geq n_0, 0 \leq c \cdot g(n) \leq f(n) \,\}$$

Relazione tra Omega e O

Possiamo affermare:

$$f(n) = O(g(n)) \iff g(n) = \Omega(f(n))$$

Notazione Big Theta

La notazione Θ (**Big Theta**) denota **limiti superiori e inferiori** (**tight bound**):

$$\Theta(g(n)) = \{\, f(n) \mid \exists c_1, c_2 \in \mathbb{Q} > 0, \exists n_0 \in \mathbb{N} : \forall n \geq n_0, 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \,\}$$

Teorema

Vale il seguente teorema:

$$\forall f(n), g(n), f(n) = \Theta(g(n)) \iff \begin{cases} f(n) = O(g(n)) \\ f(n) = \Omega(g(n)) \end{cases}$$

Proprietà riflessiva di O, Omega e Theta

Valgono le seguenti proprietà:

$$f(n) = O(f(n))$$
$$f(n) = \Omega(f(n))$$
$$f(n) = \Theta(f(n))$$

Proprietà di simmetria

$$f(n) = \Theta(g(n)) \iff g(n) = \Theta(f(n))$$

Tempo computazionale in notazione asintotica

L'analisi dell'insertion sort fatta precedentemente, possiamo affermare che:

- nel caso migliore, $T(n) = O(n), T(n) = \Omega(n)$, quindi $T(n) = \Theta(n)$;
- nel caso peggiore, $T(n) = O(n^2), T(n) = \Omega(n^2)$, quindi $T(n) = \Theta(n^2)$;
- nel caso medio, $T(n) = O(n^2), T(n) = \Omega(n)$.

Notazioni Little o, little omega

La notazione o (**little o**) denota un **limite superiore non asintoticamente stretto** (o preciso).

La notazione ω (**little omega**) denota un **limite inferiore non asintoticamente stretto** (o preciso)

Ad esempio, $2n = O(n^2) \land 2n \neq \Theta(n^2) \implies 2n = o(n^2)$. Invece, $2n^2 = O(n^2) \land 2n^2 = \Theta(n^2) \implies 2n^2 \neq o(n^2)$.

Quindi possiamo dire che:

$$o(g(n)) = \{\, f(n) \mid \forall c \in \mathbb{Q} > 0, \exists n_0 \in \mathbb{N}, \forall n \geq n_0, 0 \leq f(n) < c \cdot g(n) \,\}$$

E quindi vale la proposizione:

$$f(n) = o(g(n)) \iff \lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$$

Mentre definiamo $\omega(g(n))$:

$$\omega(g(n)) = \{\, f(n) \mid \forall c \in \mathbb{Q} > 0, \exists n_0 \in \mathbb{N}, \forall n \geq n_0, 0 \leq cg(n) < f(n) \,\}$$

E quindi vale la proposizione:

$$f(n) = \omega(g(n)) \iff \lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$$

Proprietà di simmetria trasposta

Vale la proprietà:

$$\begin{aligned} f(n) = O(g(n)) &\iff g(n) = \Omega(f(n)) \\ f(n) = o(g(n)) &\iff g(n) = \omega(f(n)) \end{aligned}$$

Proprietà transitiva

Valgono le proprietà:

$$\begin{aligned} f(n) = \Theta(g(n)) \land g(n) = \Theta(h(n)) &\implies f(n) = \Theta(h(n)) \\ f(n) = O(g(n)) \land g(n) = O(h(n)) &\implies f(n) = O(h(n)) \\ f(n) = \Omega(g(n)) \land g(n) = \Omega(h(n)) &\implies f(n) = \Omega(h(n)) \\ f(n) = o(g(n)) \land g(n) = o(h(n)) &\implies f(n) = o(h(n)) \\ f(n) = \omega(g(n)) \land g(n) = \omega(h(n)) &\implies f(n) = \omega(h(n)) \end{aligned}$$

Non sempre due funzioni sono comparabili asintoticamente.